1. 8 QUEENS PROBLEM

EXP NO: 1

Date: 29.7.26

Aim:

TO solve the 8 queens problem using Back tracking algorithm.

Algorithm:

1. Start with the first column
2. Place a queen in a safe row
3. move to next column
4. If no safe position exist, back track to previous column and done the previous queen.
5. Repeat until all 8 queen are placed.

Initial state:

An empty 8 × 8 chessboard represent as a 1D array of [-1] Q, Q 2D matrix of 0s.

Goal state:

A board containing 8 queens such as two queens attack each other (no two queens share the same row, column, or diagonal.

Objective:

place one by one queen

Guidelines: Algorithm

one by one queen which

can't attack each other.

Problem Statement:

The goal is to

place eight queens on a 8×8

chessboard so that no two

queens can attack each other.

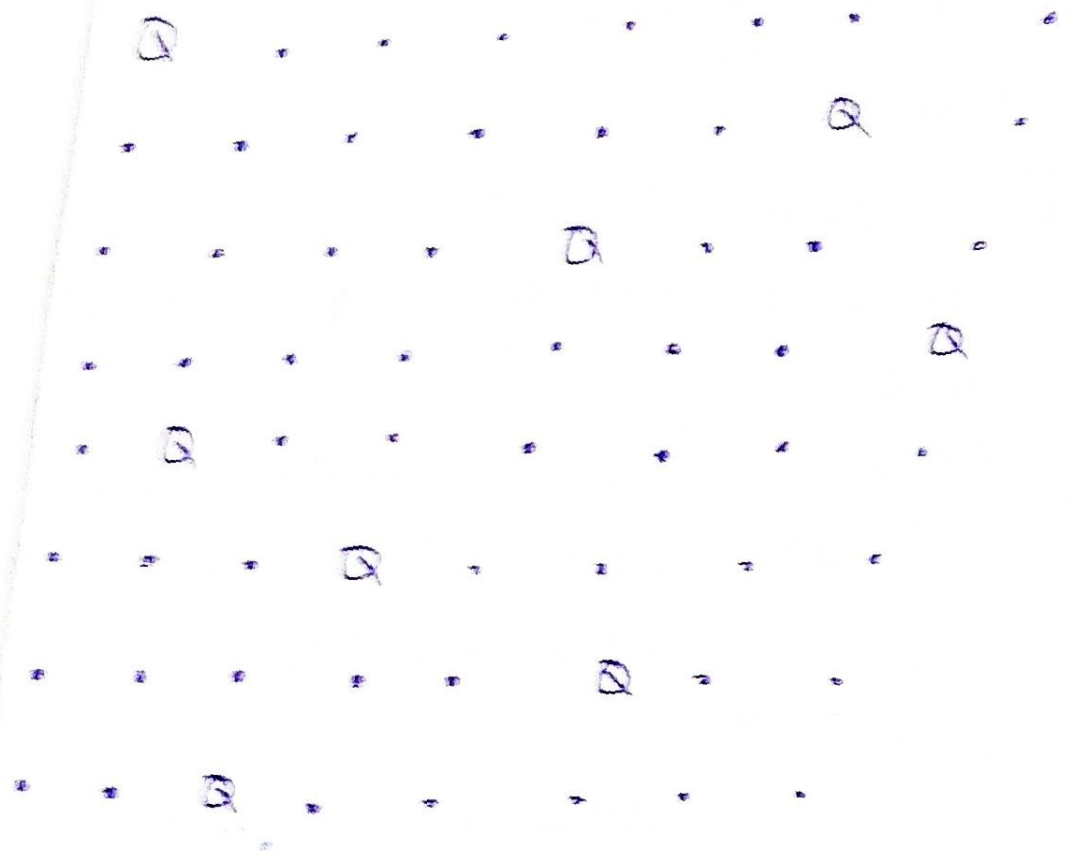
Function use:

```
def is_safe(board, row, col):
    for i in range(col):
        if board[i] == row or
        abs(board[i] - row) == abs(i - col):
            return False
    return True

def solve_8_queens(board, col):
    if col == 8:
        return True
    for row in range(8):
        if is_safe(board, row, col):
            board[col] = row
```

```
        if solve_8_queens(board, col+1):
            return True:
        board[col] = -1
    return false
def print_board(board):
    for row in range(8):
        line = "  "
        for col in range(8):
            if board[col] == row:
                line += "Q "
            else:
                line += ". "
        print(line)
initial_state = [-1, -1, -1, -1, -1, -1, -1, -1]
if solve_8_queens(initial_state, 0):
    Print("solution found:\n")
    Print_board(initial_state)
else:
    Print("no solution exist")
```

Output:



Result:

Thus the placing of 8 Queens in the matrix by using back tracking is solved and output is verified.

2. WATER JUG PROBLEM

Exp No: 2
Date: 5-9-2016

Aim:

To solve the water jug problem by finding a sequence of operations that results in the required amount of water in one of the jugs.

Problem statement:

Two jugs are given:

* Jug A = 4 litres
* Jug B = 3 litres

Initially both jugs are empty. The objective is to obtain 2 litres of water in 4 litre jug using the following operations.

* Fill a jug completely
* Empty a jug completely
* Pour water from one jug to another until either the source jug becomes empty or the destination jug becomes full.

Initial State

(0, 0)

* Jug A = 0 litres
* Jug B = 0 litres

Goal State

* Jug A contains 2 litres

Operators

The following operations are allowed

* Fill jug A
* Fill jug B
* Empty jug A
* Empty jug B
* pour water from jug A to jug B
* Pour water from jug B to jug A

Successor function

The successor function generates all valid next states from the current state by applying the available operators

Eg: 2 State (A, B):

* Fill A → (4, B)
* Fill B → (A, 3)
* Empty A → (0, B)
* Empty B → (A, 0)

* Pour A → B
* Pour B → A

Algorithm

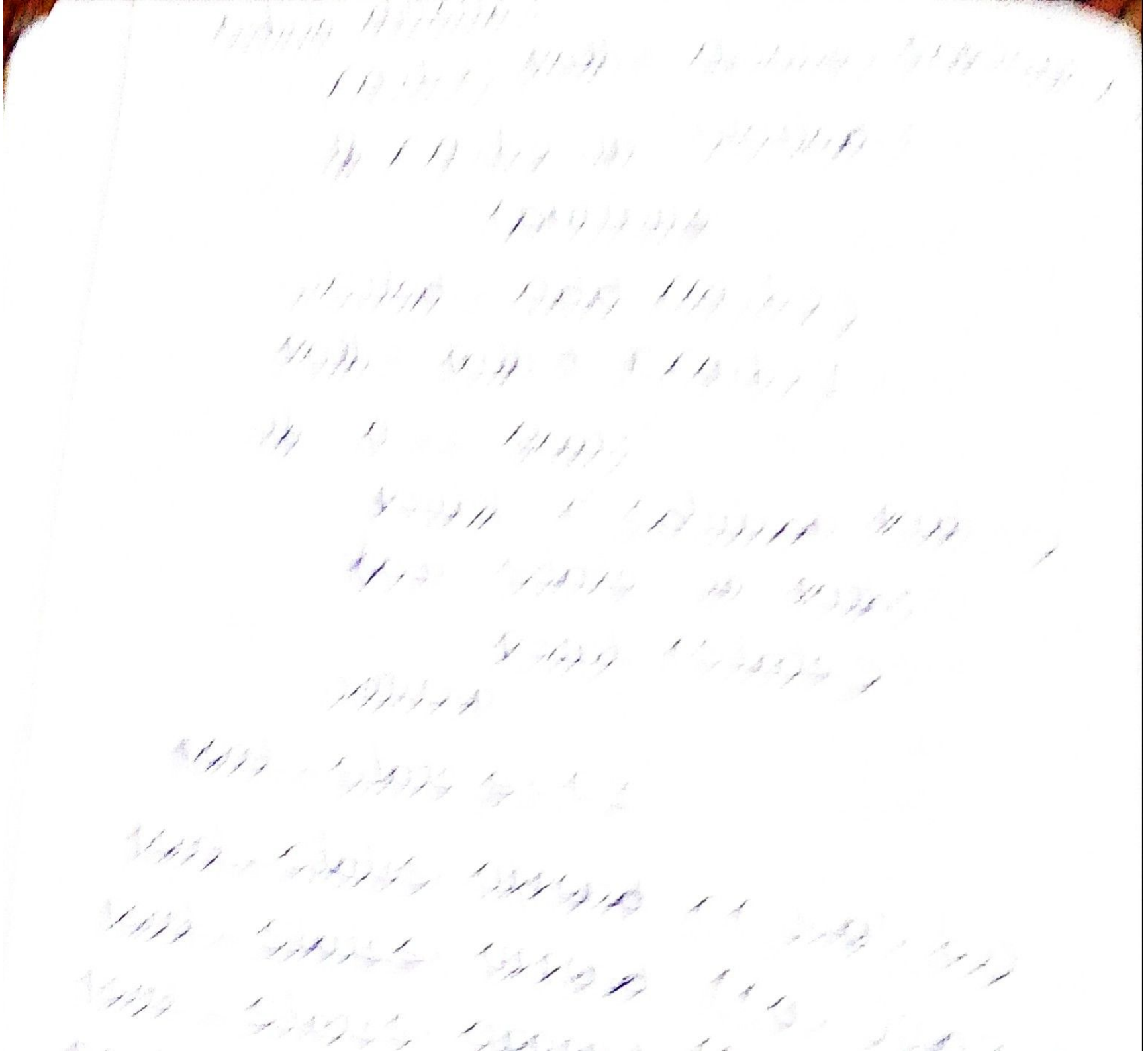
1. Start with both jugs empty (0,0)
2. Store the initial state in queue
3. Mark visited states
4. Generate all possible successor states using six operators.
5. Check whether the goal state is reached
6. If not, continue exploring new states
7. Stop when the goal state is found
8. Display the sequence of states.

Program Code

```
from collections import deque

def water_jug():
    jug1 = 4
    jug2 = 3
    goal = 2

    visited = set()
    queue = deque()
    queue.append((0,0,[]))
```



```
for state in next_states:
    if state not in visited:
        queue.append((state, path))

water_jug()
```

Output:

Solution path:

(0,0)
(0,3)
(3,0)
(3,3)
(4,2)
(0,2)
(2,0)

Result:

The water jug problem successfully solved using search and the

Goal State

1 2 3
4 5 6
7 8 0

goal_state = (1,2,3,
4,5,6,
7,8,0)

operators:

* UP => move the blank space upward
* DOWN => move the blank space downward
* Left => move the blank space to left
* Right => move the blank space to right

Successor function

For the 8-puzzle problem the current state is represented as 3x3 matrix containing tiles numbered 1 to 8 and one blank space represented by 0.

Algorithm

1. Start with the initial state
2. Create an empty stack
3. Push the initial state into stack

- [illegible]
- [illegible]
- [illegible]
- [illegible]
- [illegible]

5. [illegible]
6. [illegible]
7. [illegible]

Program Code:

```
[illegible]
```

```
for dr, dc, move in moves:
    new_row = row + dr
    new_col = col + dc
    if 0 <= new_row < 3 and
       0 <= new_col < 3:
        new_index = new_row * 3 +
                    new_col
        new_state = list(state)
        new_state[blank_index],
        new_state[new_index] =
            new_state[new_index],
            new_state[blank_index]
        successors.append((tuple(new_
                                 state),
                           move))
return successors

def dfs(start, goal):
    stack = [(start, [])]
    visited = set()
    while stack:
        current, path = stack.pop()
        if current in visited:
            continue
        visited.add(current)
```

```
current = initial_state
print("In solution path:")
print_state(current)
for move in solution:
    successors = get_successors(current)
    for successor, successor_move in
                                successors:
        if successor_move == move:
            current = successor
            break
    print("move:", move)
    print_state(current)
else:
    print("No solution found.")
```

OUTPUT:

Initial State:

1 2 3
4 0 6
7 5 8

Goal State:

1 2 3
4 5 6
7 8 0

Solution Found!

Solution path:

1 2 3
4 0 6
7 5 8

move: Down

1 2 3
4 5 6
7 0 8

move: Right

1 2 3
4 5 6
7 8 0

Result:

Problem solved

that the 8-Puzzle was successfully solved by the Depth first search and the output